Sprawozdanie z Laboratorium
Algorytmy Populacyjne w problemie TSP

Krzysztof Zając

17 czerwca 2026

1 Cel i zakres zadania

Celem zadania było zaprojektowanie i zaimplementowanie algorytmu genetycznego dla problemu komiwojażera (TSP). Algorytmy populacyjne, w przeciwieństwie do algorytmów przeszukiwania lokalnego, operują na całym zbiorze rozwiązań zwanym populacją, a ich działanie jest inspirowane biologicznym procesem doboru naturalnego i ewolucji.

Zaimplementowany algorytm rozszerzono o dodatkowe modyfikacje podnoszące skuteczność:

- Porównanie co najmniej dwóch różnych metod krzyżowania (OX1 oraz PMX).
- Zastosowanie algorytmu wyspowego.
- Zrównoleglenie obliczeń na poziomie programu z użyciem wielowątkowości i polityki wykonania w C++ (`std::execution::par_unseq`).
- Zastosowanie algorytmów memetycznych (Lamarckian learning poprzez wbudowany 2-opt).

2 Architektura Algorytmu Genetycznego

Ogólny schemat algorytmu genetycznego składa się z wyznaczenia populacji początkowej, ewaluacji, selekcji, krzyżowania i mutacji. Poszczególne kroki zrealizowano w następujący sposób:

- **Reprezentacja i Populacja początkowa:** Osobnik (genotyp) jest reprezentowany jako ciąg kodowy genów (permutacja wierzchołków). Zgodnie z założeniami, populacja początkowa generowana jest losowo.
- **Ewaluacja i Selekcja:** Wartość liczbowa odpowiadająca genotypowi (fenotyp) to funkcja celu – całkowity dystans trasy. Zastosowano metodę selekcji turniejowej. Zapewnia ona wybór najlepszego osobnika z wylosowanej podgrupy.
- **Krzyżowanie:** Zaimplementowano operatory Order Crossover (OX1) oraz Partially Mapped Crossover (PMX) w celu weryfikacji ich skuteczności. Są to specjalistyczne procedury zachowujące dopuszczalność rozwiązań, czyli to, że ciąg potomny pozostaje poprawną permutacją. Zawsze wybierani są dwa różni rodzice w celu wygenerowania potomków.

- **Mutacja:** Zgodnie z wytycznymi z zadania, proces złożenia permutacji zastąpiono lepszą metodą: mutacja polegała na losowej zamianie dwóch wierzchołków w chromosomie (swap). Zabieg ten stosowano z prawdopodobieństwem wynoszącym 10% w celu utrzymania różnorodności genetycznej.
- **Algorytm Memetyczny:** Wykorzystany algorytm nie bazuje wyłącznie na genach przekazanych od rodziców, ale na „memach"(informacja zdobyta w środowisku). W ramach etapu "dorastania/uczenia się", każdy osobnik po krzyżowaniu podlega wbudowanej procedurze przeszukiwania lokalnego 2-opt.
- **Model Wyspowy:** Zastosowano architekturę odizolowanych od siebie populacji (wysp). Zaimplementowano 4 sub-populacje, pomiędzy którymi migracja najlepszych osobników w zamian za usunięcie najsłabszych następuje co 10 pokoleń.
- **Warunek stopu:** Algorytm w każdym teście zatrzymywał się po wykonaniu sztywnej, stałej liczby iteracji (pokoleń), zdefiniowanej na poziomie epok.

3 Wyniki eksperymentów dla Algorytmu Genetycznego

Zgodnie z wymaganiami podano średnie wyniki uzyskane dla 100 wykonań algorytmu (trials). Badania przeprowadzono dla instancji `qa194.tsp`. Zastosowano maksymalną liczbę iteracji (epok) równą 100.

Wariant	**Krzyżowanie**	**Najlepszy [j]**	**Średni [j]**	**Czas [ms]**
Bez wysp (1 pop.)	OX1	9 352	9 359.96	118 349
Bez wysp (1 pop.)	PMX	9 352	9 365.60	102 187
4 Wyspy	OX1	9 352	9 357.63	75 713
4 Wyspy	PMX	9 352	9 362.41	71 743
Brak procedury memetycznej*	**PMX**	**44 986**	**50 931.8**	**552**

Tabela 1: Wyniki algorytmu memetycznego dla instancji qa194 po 100 uruchomieniach. *Wariant testowy obrazujący wpływ wyłączenia intensywnego przeszukiwania lokalnego.

Wnioski z działania GA oraz anomalia wydajnościowa: Zastosowanie silnego algorytmu memetycznego poskutkowało niezwykłą stabilnością rozwiązań – średnia długość trasy niemal w ogóle nie odbiega od wartości najlepszej. Algorytm odnajdował wysoce optymalny układ za każdym razem. Operator OX1 okazał się minimalnie lepszy jakościowo od PMX. Implementacja 4 niezależnych wysp pozwoliła podnieść średnią jakość (dzięki uniknięciu przedwczesnej zbieżności) i drastycznie przyspieszyła ewolucję (spadek czasu ze 118s do ok. 75s).

Na szczególną uwagę zasługuje jednak dodatkowy eksperyment z wyłączoną/zredukowaną procedurą memetyczną (ostatni wiersz tabeli). Czas wykonania spadł drastycznie – z 71 sekund do zaledwie 552 ms, jednak jakość rozwiązań uległa kompletnej degradacji (wynik rzędu 44 986). Pokazuje to niezwykle fascynującą zależność: czysty algorytm genetyczny (oparty wyłącznie na krzyżowaniu i losowych mutacjach) radzi sobie z problemem komiwojażera bardzo słabo i wolno zbiega. Prawdziwą siłą zaimplementowanego rozwiązania jest ulepszenie Lamarcka – ciężkie i kosztowne obliczeniowo przeszukiwanie otoczenia wbudowane w ewolucję osobników.

4 Zestawienie wyników z całego semestru

W celu pełnego obrazu ewaluacji w sprawozdaniu zestawiono wyniki i skuteczność wszystkich algorytmów zaimplementowanych podczas całego semestru, weryfikowanych na grafie `qa194.tsp`.

Algorytm	Najlepszy wynik	Średni wynik	Czas wyk. (całość) / Uwagi
Losowe próbkowanie	83 114	∼85 659	Błyskawiczny, brak optymalizacji
Czysty Algorytm Genetyczny (bez 2-opt)	44 986	50 931.8	552 ms (100 prób)
Aproksymacja 2-opt (MST-DFS)	12 446	-	Wykonanie deterministyczne
Local Search (Pełne otoczenie)	9 877	10 125	Zbieżność w ok. 210 krokach (∼142 ms)*
Local Search (Próbkowanie)	14 717	15 857	Utyka w minimach lokalnych (∼142 ms)*
Local Search (Inicjalizacja MST)	9 824	9 920	B. szybki, zbieżność w 54 krokach (∼142 ms)*
Simulated Annealing (SA)	9 656	10 065.7	1 031 ms (100 prób)
Tabu Search (TS)	9 787	10 279.5	265 ms (100 prób)
Algorytm Memetyczny (Wyspy, OX1)	**9 352**	**9 357.6**	∼75 000 ms (100 prób)

Tabela 2: Porównanie skuteczności badanych metaheurystyk na instancji qa194. *Czas 142 ms dotyczy łącznego wykonania wszystkich trzech wariantów Local Search.

Końcowe wnioski porównawcze:

1. **Jakość rozwiązań:** Metoda populacyjna połączona z algorytmem memetycznym całkowicie deklasuje poprzednie warianty. Znalezione rozwiązanie rzędu 9 352 jest zauważalnie lepsze zarówno od algorytmów przeszukiwania w oparciu o pojedynczą trajektorię (SA - 9 656, TS - 9 787), jak i podstawowych wariantów Local Search.

2. **Złożoność czasowa vs Jakość:** Klasyczne heurystyki Local Search są nieprawdopodobnie szybkie – wykonanie wszystkich wariantów zajęło łącznie zaledwie 142 ms. Ekstremalny wzrost jakości w Algorytmie Genetycznym opartym na Lamarcku kosztuje drastyczny narzut obliczeniowy (75 sekund). Jak udowodnił dodatkowy eksperyment, wyłączenie tej logiki zbija opóźnienia do ułamków sekund (552 ms), ale pozostawia wynik trzykrotnie gorszy niż głupie, losowe przeszukiwanie lokalne (44 986 vs 14 717).

3. **Wykorzystanie wiedzy problemowej:** Wprowadzenie optymalizacji *a priori* daje świetne rezultaty niezależnie od metody. W Local Search start z silnego punktu (MST o wadze 8 026) zredukował średnią liczbę kroków do zbieżności z 210 aż do 54. W algorytmach ewolucyjnych odpowiednikiem użycia wiedzy problemowej jest właśnie wbudowany mechanizm memetyczny uczący się otoczenia.

Rekomendacja dla dużych zbiorów danych:
Podsumowując, wybór optymalnego algorytmu dla bardzo dużych instancji problemu TSP staje się problemem klasycznego inżynieryjnego kompromisu między opóźnieniami w wykonywaniu procedur a ostateczną precyzją. Zastosowany Algorytm Memetyczny zapewnia doskonałe rezultaty dla mniejszych grafów (`qa194`), jednak jego drastyczna złożoność czasowa sprawia, że skaluje się on bardzo słabo. Wynika to z faktu, że dla każdej z G epok i dla każdego z P osobników w populacji wykonujemy wbudowany algorytm Local Search (2-opt), którego złożoność pojedynczego przeszukania pełnego otoczenia wynosi $O(N^2)$. Daje to całkowitą złożoność rzędu $O(G \cdot P \cdot k \cdot N^2)$, gdzie k to liczba kroków do zbieżności danego osobnika.

Dlatego w perspektywie potężnych zbiorów danych zdecydowanym zwycięzcą okazuje się **Symulowane Wyżarzanie (SA)** połączone ze strojeniem parametrów metodą *a*

priori. Jak udowodniły to poprzednie eksperymenty, SA potrafi z wysoką skutecznością wyjść z minimów lokalnych, jednocześnie zachowując czas wykonania na poziomie pojedynczych sekund, ponieważ jego złożoność to wysoce skalowalne $O(I)$. Z kolei tam, gdzie absolutnym priorytetem w systemie jest najniższe opóźnienie z sensownym przybliżeniem, bezkonkurencyjny pozostaje **Local Search z inicjalizacją za pomocą Minimalnego Drzewa Rozpinającego (MST)**. Ogranicza on liczbę kosztownych kroków wspinania się na wzgórze do minimum, prowadząc do zbieżności i akceptowalnych wyników w ułamkach sekund.